

# Smoker Software Design Description

**Implementation baseline:** 30 August 2026 (development tree; documentation reconciliation) **Status:** Reconciled with current implementation and completed 61,035-run forensic audit

## 1. Purpose

Smoker is a Docker-based framework for testing CPAN modules under controlled dependency configurations.

Its purpose is not merely to determine whether a CPAN module passes or fails. Smoker is designed to preserve enough evidence to investigate how specified dependency versions affect module behavior across different Perl versions.

Each test case is executed independently in a Docker container. The resulting status, return code, logs, metadata, and supporting artifacts are retained for later examination.

The central execution invariant is:

**One unique plan row -> one run directory -> one rc.code -> one summary row**

This invariant establishes execution and result-accounting integrity. It does not by itself establish the ultimate causal interpretation of a module failure. Later confirmation testing may refine that interpretation without altering the historical result.

## 2. Scope

Smoker supports three principal testing modes:

- **baseline** — test the target module without deliberately varying dependency versions.
- **vary-one** — install one specified dependency version before testing the target.
- **vary-two** — install two specified dependency versions before testing the target.

Tests may be executed across multiple Perl Docker base images.

The framework is responsible for:

- reading and validating test plans;
- deduplicating equivalent plan rows;
- allocating a permanent run directory for every dispatched test;

- controlling concurrent execution;
- constructing the requested dependency environment;
- running the target module installation/test operation;
- preserving execution evidence;
- classifying the result;
- maintaining the batch summary.csv;
- recovering interrupted or incomplete worker results where possible;
- independently validating completed batches.

Smoker is an empirical testing framework. It does not claim that every observed failure proves a dependency incompatibility. Failure interpretation depends on the evidence preserved by the run and, where necessary, later confirmation testing.

### 3. Plan Identity

A test is defined by its effective plan fields, including:

- Docker base image;
- mode;
- target module;
- target version;
- first dependency and requested version, when applicable;
- second dependency and requested version, when applicable.

Equivalent definitions are deduplicated before execution.

For vary-two, dependency pairs are treated as unordered for plan-identity purposes. Reversing the two dependency columns does not create a distinct test.

The scheduler assigns a unique run identity and permanent run directory to each admitted unique plan row.

### 4. Architecture

The principal implementation components are:

`Run_Smoker.pl`

Top-level launcher.

Responsibilities include:

- establishing the Smoker environment;
- checking backup configuration;
- optionally running the pre-test backup;
- starting the CSV scheduler;

- managing the scheduler process group;
- handling launcher-level interruption and shutdown;
- invoking batch validation after execution.

`bin/run_plan_csv.pl` and `lib/Smoker/Scheduler.pm`

The command-line entry point delegates batch scheduling and worker coordination to `Smoker::Scheduler`.

Responsibilities include:

- reading the CSV plan;
- validating and deduplicating plan definitions;
- establishing the batch result directory;
- controlling worker concurrency;
- reserving run directories before workers begin;
- mapping worker PIDs to plan rows and run directories;
- reaping workers;
- recovering abnormal worker termination;
- finalizing active and undispatched rows during scheduler interruption.

`lib/Smoker/Runner.pm`

Per-test execution coordinator.

Responsibilities include:

- accepting or allocating the permanent run directory;
- coordinating dependency and target execution;
- interpreting ordinary execution outcomes;
- invoking shared result finalization;
- converting internal framework exceptions into evidenced framework failures where possible.

`lib/Smoker/Inner.pm`

Container-side execution logic.

Responsibilities include the requested dependency installation and target module installation/test operation.

Requested dependency versions are compared using Perl version semantics. If a module exposes no package VERSION, Smoker logs that local verification is inconclusive and continues; this is not proof of an exact installed version.

`lib/Smoker/Docker.pm`

Docker execution layer.

Responsibilities include container invocation, command execution, timeout handling, and preservation of low-level process evidence.

`lib/Smoker/Result.pm`

Result classification and finalization layer.

Responsibilities include:

- writing `rc.code`;
- writing `result.meta`;
- maintaining execution evidence;
- maintaining `summary.csv`;
- enforcing framework-failure contracts;
- performing summary update-or-insert and reconciliation.

The summary function is named:

`_update_or_insert_summary_row`

`bin/validate_smoker_batch.pl`

Independent batch validator.

It evaluates the resulting batch from persisted artifacts rather than accepting the producer's own claims of completeness or correctness.

## 5. Run Lifecycle

For each unique admitted plan row:

1. The scheduler reserves a permanent run directory.
2. A worker is associated with that exact plan row and directory.
3. The requested Docker execution environment is established.
4. Requested dependency versions are applied according to test mode.
5. The target module is installed/tested.
6. Execution evidence is retained.
7. The result is classified.
8. `rc.code` is written atomically through the shared finalizer.
9. `result.meta` is completed.
10. Exactly one current `summary.csv` row is established for the run.
11. The scheduler reaps the worker and verifies or repairs incomplete framework state when necessary.

The reserved run directory is not replaced merely because execution fails.

This permits the parent scheduler to recover abnormal worker termination into the same execution identity rather than inventing a replacement test.

## 6. Result Artifacts

A normal run directory preserves the evidence needed to understand the execution outcome.

Principal artifacts include:

- `build.log`
- `run.err`
- `rc.code`
- `result.meta`
- execution-trail information
- framework evidence when the failure is internal to Smoker

Additional artifacts may be present depending on the execution path.

`rc.code` contains the normalized integer result code.

`result.meta` records structured classification information and supporting state.

The batch-level `summary.csv` provides one current summary row for each unique executed plan definition.

The persisted artifacts, rather than terminal output alone, are the authoritative evidence for later validation and analysis.

## 7. Summary Maintenance

`summary.csv` is shared by concurrent workers and therefore requires serialized modification.

Smoker uses a summary lock together with a batch-local run-ID membership index and per-run transaction state.

The implementation provides two paths.

### 7.1 Ordinary First-Time Insertion

For an ordinary first-time result where:

- the per-run summary transaction state is empty;
- reconciliation has not been requested; and
- the run ID is not already represented in the membership index,

Smoker uses an optimized append path.

The sequence is:

1. atomically mark the per-run summary state pending;
2. append the new row to `summary.csv`;

3. atomically create/update the run-ID index entry;
4. atomically mark the per-run summary state committed.

This avoids parsing and rewriting the entire historical summary for every ordinary first-time result.

## 7.2 Update-or-Insert Reconciliation

When retry, recovery, damaged transaction state, or an existing run-ID entry requires reconciliation, Smoker reads the summary and performs a full update-or-insert operation.

For the requested `run_id` it:

- retains or replaces the appropriate current row;
- removes duplicate rows for that run ID;
- inserts the requested row if no matching row exists;
- writes the reconstructed summary to a temporary file;
- atomically replaces the previous summary;
- updates the run-ID index;
- marks the per-run transaction state committed.

This makes repeated finalization and recovery idempotent with respect to the summary row.

## 7.3 Legacy Summary Bootstrap

If the run-ID membership index has not yet been initialized, Smoker scans the existing summary once to establish the index.

After initialization, ordinary first-time insertions do not require parsing all previous summary rows.

A missing, empty, pending, or otherwise damaged per-run transaction state causes conservative reconciliation rather than assuming that a prior partial operation completed successfully.

## 8. Result Policy

The principal public result states are:

### PASS

`rc=0`

The requested test completed successfully.

### UNAVAILABLE

`rc=111`

The requested test could not be meaningfully executed because required test material was unavailable, such as a requested exact release that could not be obtained.

## FAIL

A nonzero failure result other than the defined UNAVAILABLE condition.

Module/dependency failures retain their applicable execution return code. Raw codes 124, 137, and 143 do not by themselves prove timeout. Only explicit Docker timeout state normalizes the result to rc=8; log-limit termination uses rc=112.

The current Inner layer emits confirmed unavailability as rc=111. The shared classification policy also normalizes rc=4 to rc=111 when explicit confirmation evidence is supplied; otherwise rc=4 remains FAIL,4.

## Framework failure

rc=125

Internal Smoker failures are represented as:

- status FAIL;
- rc 125;
- subtype framework\_error;
- a nonempty causal reason;
- explicit framework evidence.

The shared finalizer rejects inconsistent framework classifications. For example, rc=125 without framework\_error, or framework\_error with a non-125 return code, is not accepted as a valid final result.

Framework failures are kept distinct from failures caused by the module or requested dependency environment.

## 9. Scheduler Recovery

The parent scheduler retains enough information to associate each active worker with:

- its PID;
- its plan row;
- its already-reserved run directory.

If a worker terminates abnormally or leaves incomplete result state, the scheduler attempts to recover the test as an evidenced framework failure in that same run directory.

Recovery covers conditions including:

- missing or malformed `rc.code`;
- missing or malformed `result.meta`;
- missing or duplicate summary rows;
- inconsistent summary status or return code;
- incomplete framework evidence;
- child exceptions outside the normal Runner boundary;
- signal termination;
- fork failure after run-directory reservation.

Repeated recovery must not create duplicate summary rows.

Already complete PASS, module FAIL, UNAVAILABLE, or properly evidenced framework results are preserved rather than overwritten merely because the scheduler examines them during recovery.

## 10. Interruption Handling

SIGINT and SIGTERM are handled so that an interrupted batch retains explicit accounting for admitted work.

The scheduler finalizes active and undispatched admitted rows as evidenced framework failures when normal execution cannot continue.

At the launcher level, the scheduler process group is first sent the applicable termination signal. The launcher allows a grace period of approximately five seconds. Remaining processes may then be killed, and the group leader is reaped.

This is intended to prevent a launcher interruption from silently leaving admitted tests without persistent result accounting.

A parent scheduler crash from which the scheduler itself cannot execute its recovery path remains outside the guarantees of normal in-process recovery.

## 11. Independent Validation

Completed batches are evaluated by `bin/validate_smoker_batch.pl`.

The validator independently examines persisted result evidence rather than trusting producer-side declarations.

Its principal checks include:

### Execution accounting

Every unique admitted plan definition must correspond to the expected execution path.

## Required-result completeness

Required result artifacts must exist and be internally usable.

## Classification integrity

The persisted status and return code must agree with the available evidence and classification rules.

## Summary/run-directory consistency

Summary rows must correspond to the correct run directories and result artifacts.

## Framework evidence

Framework failures must contain the evidence required by the framework-failure contract.

The validator is deliberately capable of rejecting corrupted result sets even when producer-side metadata claims that they are complete or valid.

Validation therefore provides an independent check on Smoker's own result-production logic.

## 12. Backup And MiniCPAN Workflow

Before a test batch begins, `Run_Smoker.pl` may offer to run the Smoker backup utility.

The interactive prompt is:

Run pre-test backup? [y/N]

The default is **No**.

Behavior is:

- y or Y — run the pre-test backup;
- Enter — skip;
- n or N — skip;
- invalid input — skip;
- prompt timeout — skip;
- EOF — skip;
- noninteractive execution — skip.

The normal prompt timeout is 10 seconds. A test-specific environment override is available for automated testing.

The backup utility is `bin/bksmoker.sh`.

The backup script itself does not issue the launcher prompt.

Backup facilities may preserve Smoker state under /QuickBackup and may include MiniCPAN or other configured material according to the local backup configuration.

Smoker may use a local MiniCPAN when a usable package index is present. Otherwise the framework may fall back to other configured CPAN access.

Tarball caching is used to reduce unnecessary repeated retrieval.

The availability of an archive in a local mirror is not, by itself, treated as proof that a requested release exists or does not exist on CPAN. Later forensic confirmation may distinguish local-mirror absence from true release unavailability.

## 13. Operational Boundaries

Smoker provides strong accounting for work admitted and handled through its normal launcher/scheduler execution paths.

The guarantees have defined boundaries.

In particular:

- host failure can terminate processes outside Smoker's recovery control;
- Docker or operating-system failure may prevent normal finalization;
- storage failure may make persistent recovery impossible;
- scheduler-parent destruction that prevents recovery code from running is not equivalent to a handled scheduler interruption;
- external CPAN state can change over time;
- historical availability or failure observations therefore require careful interpretation.

The framework preserves evidence so that such distinctions can be investigated rather than hidden.

## 14. Large-Run Validation And Interpretation

A major validation exercise executed **61,035 test cases across five Perl versions**.

The historical batch recorded:

- **18,398 PASS**
- **31,077 FAIL**
- **11,560 UNAVAILABLE**

The unexpectedly large FAIL count was not accepted at face value as evidence of 31,077 currently reproducible module or dependency failures.

Instead, the original batch was preserved unchanged and subjected to a subsequent forensic audit involving:

- exact historical target confirmation;
- dependency-version confirmation across Perl bases;
- configuration-failure confirmation;
- missing-prerequisite confirmation;
- target-download confirmation;
- timeout investigation;
- dependency-did-not-remain sampling;
- local-MiniCPAN versus live-CPAN availability checks;
- structural result validation;
- deterministic regeneration of the final evidence overlays.

The forensic process did **not rewrite the historical summary**.

Instead, later evidence was associated with the historical rows to distinguish such outcomes as:

- reproduced failures;
- version-dependent failures;
- exact releases now confirmed unavailable;
- changed failure mechanisms;
- failures that did not reproduce under confirmation conditions;
- sample-supported historical mechanisms.

All 61,035 historical rows are now accounted for either by their original non-failure status or by a resolved causal recalibration category.

Evidence strength is explicitly distinguished.

Some conclusions are based on direct reruns of exact historical cases. Others are based on repeated confirmation of an exact dependency/version or configuration pair across all five Perl bases. The historical dependency-did-not-remain population is supported by an exact-history diagnostic sample and is therefore described as sample-supported rather than as though all 5,694 historical rows were individually rerun.

The detailed methodology, counts, limitations, and evidence assignments are documented in:

`Smoker_61035_Final_Forensic_Audit_Report.md`

The audit demonstrates an important design distinction:

## **Execution integrity and causal interpretation are related but separate questions.**

Smoker's result architecture establishes what test was admitted, what execution path occurred, what artifacts were produced, and what result was recorded. Subsequent empirical evidence may refine why that historical result occurred without changing the historical record itself.

## **15. Design Boundary**

Smoker's core invariant remains:

**One unique plan row -> one run directory -> one rc.code -> one summary row**

The architecture is designed to make violations of that invariant detectable and, for handled worker and scheduler failures, recoverable.

Independent validation provides a second layer of evidence that the execution accounting and result artifacts are structurally coherent.

Neither the invariant nor the validator is intended to turn an observed module failure automatically into a causal claim about a dependency.

The preserved evidence, combined with targeted confirmation where necessary, supports that later analysis.

Historical results therefore remain immutable while later validation and forensic evidence may refine their interpretation.